

Artificial Intelligence

Practical Solutions

Practical 1: Write a PROLOG program to implement the family tree and demonstrate the family relationship.

Code:

```
female(pam).  
female(liz).  
female(pat).  
female(ann).  
male(jim).  
male(bob).  
male(tom).  
male(peter).  
parent(pam,bob).  
parent(tom,bob).  
parent(tom,liz).  
parent(bob,ann).  
parent(bob,pat).  
parent(pat,jim).  
parent(bob,peter).  
parent(peter,jim).  
mother(X,Y):-  
    parent(X,Y),  
    female(X).  
father(X,Y):-  
    parent(X,Y),  
    male(X).  
haschild(X):-  
    parent(X,_).
```

sister(X,Y):-

parent(Z,X),

parent(Z,Y),

female(X),

X\==Y.

brother(X,Y):-

parent(Z,X),

parent(Z,Y),

male(X),

X\==Y.

grandparent(X,Y):-

parent(X,Z),

parent(Z,Y).

grandmother(X,Z):-

mother(X,Y),

parent(Y,Z).

grandfather(X,Z):-

father(X,Y),

parent(Y,Z).

wife(X,Y):-

parent(X,Z),

parent(Y,Z),

female(X),

male(Y).

uncle(X,Z):-

brother(X,Y),

parent(Y,Z).

predecessor(X, Z) :-

parent(X, Z).

predecessor(X, Z) :-

parent(X, Y),

predecessor(Y, Z).

Practical 2: Write a Prolog program to implement conc (L1, L2, L3) where L2 is the list to be appended with L1 to get the resulted list L3.

Code:

```
conc(L1,[],L1):-!.  
conc([],[],[]):-!.  
conc([],L2,L2):-!.  
conc([H|L1],L2,[H|L3]):-conc(L1,L2,L3).
```

Practical 3: Write a Prolog program to implement reverse (L, R) where List L is original and List R is reversed list.

Code:

```
conc([],L,L).  
conc([H|T],L2,[H|R]):-conc(T,L2,R).  
reverse([],[]).  
reverse([H|T],R):-reverse(T,R1),conc(R1,[H],R).
```

Practical 4: Write a Prolog program to calculate the sum of two numbers.

Code:

```
sum:- write('Enter first number : '),  
read(X),  
write('Enter second number : '),  
read(Y),  
S is X+Y,  
write(S).
```

Practical 5: Write a Prolog program to implement max(X, Y, M) so that M is the maximum of two numbers X and Y.

Code:

```
max(X,Y,M):-  
X>Y -> M is X; M is Y.
```

Practical 6: Write a program in PROLOG to implement factorial (N, F) where F represents the factorial of a number N.

Code:

factorial(N,F):- N=0,F is 1.

factorial(N,F):-N>0, N1 is N-1, factorial(N1,F1),F is F1*N .

Practical 7: Write a program in PROLOG to implement generate_fib(N,T) where T represents the Nth term of the fibonacci series.

Code:

fibonacci(0,1).

fibonacci(1,1).

fibonacci(N,T):- N>1,N1 is (N-1),fibonacci(N1,R1),N2 is (N-2),fibonacci(N2,R2), T is (R1+R2).

Practical 8: Write a Prolog program to implement power (Num,Pow, Ans) : where Num is raised to the power Pow to get Ans.

Code:

power(_,0,1).

power(N,1,N).

power(N,P,R):-P>0,P1 is P-1,power(N,P1,R1),R is N*R1.

Practical 9: Prolog program to implement multi (N1, N2, R) : where N1 and N2 denotes the numbers to be multiplied and R represents the result.

Code:

multiply(0,B,0).

multiply(A,0,0).

multiply(A,B,C):-A>0,B>0,R is B-1,multiply(A,R,C1),C is A+C1.

Practical 10: Write a Prolog program to implement memb(X, L): to check whether X is a member of L or not.

Code:

mem(X,[X|_]):-!.

mem(X,[_|T]):-mem(X,T).

Practical 11: Write a PROLOG program to implement `sumlist(L, S)` so that `S` is the sum of a given list `L`.

Code:

```
suml([],0):-!.  
suml([H|T],S):-suml(T,S1),S is H+S1.
```

Practical 12: Write a PROLOG program to implement two predicates `evenlength(List)` and `oddlength(List)` so that they are true if their argument is a list of even or odd length respectively.

Code:

```
leng([],0).  
leng([H|T],R):-leng(T, F1), R is F1+1.  
evenl(L):- leng(L, N), D is N mod 2, D == 0.  
oddl(L):- leng(L,N), D is N mod 2, D==1.
```

Practical 13: Write a PROLOG program to implement `maxlist(L, M)` so that `M` is the maximum number in the list.

Code:

```
maxlist([],0):-!.  
maxlist([H|T],M):-maxlist(T,R),(H>=R->M is H;M is R).
```

Program 14: Write a PROLOG program to implement `insert(I, N, L, R)` that inserts an item `I` into `N`th position of list `L` to generate a list `R`.

Code:

```
insert(I,1,L,[I|L]).  
insert(I,N,[H|T],[H|R]):- N1 is N-1,  
                           insert(I,N1,T,R).
```

Program 15: Write a PROLOG program to implement `delete(N, L, R)` that removes the element on `N`th position from a list `L` to generate a list `R`.

Code:

```
delete(1,[_|T],T).  
delete(N,[X|T],[X|R]):- N<0 ,write('Enter Valid Num');
```

N1 is N-1,
delete(N1,T,R).